Optimizing SDE Simulation: An Implementation and Complexity Analysis of Multilevel Monte Carlo

Dhwanil Shah
SRFP Paper Implementation Report

Abstract

Valuing financial options through Stochastic Differential Equations (SDEs) typically relies on standard Monte Carlo simulation, a method that suffers from a crippling computational bottleneck: achieving an accuracy of ϵ requires an execution time proportional to $O(\epsilon^{-3})$. This project implements the Multilevel Monte Carlo (MLMC) algorithm proposed by M.B. Giles (2008) to break this barrier. By simulating tightly coupled coarse and fine paths using the same underlying Brownian increments, the algorithm recursively acts as a control variate to drastically reduce variance.

This report details the end-to-end construction of a custom C++ pricing engine. Beyond validating the theoretical mathematics—such as empirically proving the strong and weak convergence rates of the Euler-Maruyama scheme—this project highlights the practical software engineering required for high-performance finance. By utilizing pre-allocated memory buffers to eliminate dynamic allocation overhead, the MLMC implementation successfully achieved the theoretical $O(\epsilon^{-2}(\log \epsilon)^2)$ complexity. Benchmark testing confirmed exponential time savings at high accuracies, reducing the execution time of a standard SDE simulation from 94 seconds to 56 seconds at $\epsilon = 0.005$, bridging the gap between theoretical mathematics and low-latency implementation.

---

1 Introduction: Two Paths to Option Pricing

In quantitative finance, valuing an option involves computing the expected value of a financial asset's future payoff. When the underlying asset is modeled as a Stochastic Differential Equation (SDE), there are two primary computational approaches to finding this expected value.

The first is the **PDE / Discretization approach**. By leveraging the Feynman-Kac theorem, the expectation of the stochastic process can be represented as a deterministic Partial Differential Equation (e.g., the Black-Scholes PDE). While this backward-solving grid method eliminates randomness and provides high accuracy, it suffers from the "curse of dimensionality"—it becomes computationally impossible to hold the grid in memory if the option depends on multiple underlying assets.

The second is the **Monte Carlo approach**. By computationally simulating the SDE thousands of times using random Brownian motion increments, we can average the final payoffs. This approach is highly flexible and avoids the dimensionality curse, making it the industry standard. However, Standard Monte Carlo suffers from a severe computational bottleneck: to achieve a target accuracy of ϵ, the execution time scales proportionally to $O(\epsilon^{-3})$. The Multilevel Monte Carlo (MLMC) method overcomes this bottleneck.

2 Theoretical Foundations & Constraints

2.1 Brownian Motion and Multi-Resolution Paths

The foundation of the MLMC approach relies on the ability to view a single Brownian path at different resolutions. This can be conceptualized using the Paley-Wiener representation of Brownian motion:

$$W(t) = Z_0 \frac{t}{\sqrt{2\pi}} + \frac{2}{\sqrt{\pi}} \sum_{n=1}^{\infty} Z_n \frac{\sin(\frac{1}{2}nt)}{n} \tag{1}$$

where $\{Z_i\}$ are independent standard normal variables. As implemented in the project codebase, truncating this infinite series at different values of M demonstrates how low-frequency terms define the macro-trend of the path, while high-frequency terms add microscopic detail.

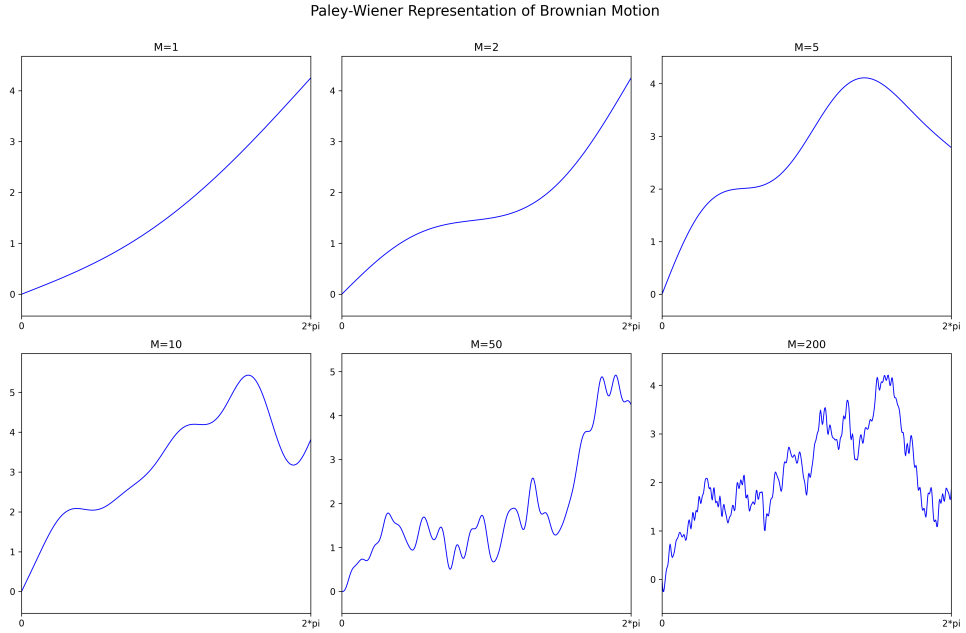

Figure 1: Empirical generation of the Paley-Wiener representation at varying resolutions.

2.2 Weak vs. Strong Convergence

To simulate the SDEs computationally, the Euler-Maruyama discretization method is utilized. Understanding the convergence of this method is critical, as MLMC relies on a mathematical mechanic that Standard Monte Carlo ignores.

- **Weak Convergence ($O(\Delta t)$):** Controls the error of the means: $|\mathbb{E}[X_{true}] - \mathbb{E}[X_{approx}]|$. Standard Monte Carlo only requires weak convergence, as it only concerns the final average of thousands of paths, regardless of how inaccurate individual simulated trajectories might be.
- **Strong Convergence ($O(\Delta t^{1/2})$):** Controls the mean of the pathwise errors: $\mathbb{E}[|X_{true} - X_{approx}|]$. This is a path-dependent property. If the exact same random Brownian increments are fed into both the true SDE and the numerical approximation, strong convergence guarantees that the physical distance between the two trajectories shrinks as the time step Δt decreases.

MLMC exploits strong convergence. By simulating a "coarse" path and a "fine" path using the *exact same* random numbers, strong convergence guarantees both paths are physically close to

the true mathematical path, and therefore, closely glued to each other. This tight path coupling drastically reduces the variance of their difference.

2.3 The Global Lipschitz Condition

The mathematical proof guaranteeing that coarse and fine payoffs remain close relies on the payoff function obeying a Global Lipschitz condition. This acts as a mathematical "speed limit," ensuring the slope of the function is strictly bounded and never becomes vertical.

While this may seem like an academic convenience, it perfectly aligns with reality for standard market instruments. European Calls and Puts have maximum slopes of exactly 1 or -1, making them perfectly Lipschitz. However, this assumption breaks down for exotic options. For example, a Digital Option has an infinite, vertical slope at the strike price, and highly non-linear interest rate SDEs (featuring faster-than-linear growth or unbounded derivatives near zero) will cause the basic Euler-Maruyama stepper to mathematically explode.

2.4 Cost Optimization and Error Balancing

In standard SDE simulation, total error is composed of statistical noise ($O(1/\sqrt{M})$) and discretization bias ($O(\Delta t)$). To ensure the total error remains below a target ϵ, it may seem mathematically valid to make one error source much smaller than the other. However, computationally, this is disastrous:

- **Over-solving Δt:** If bias is reduced to ϵ^2, almost the entire error budget goes to statistical noise. The algorithm must take incredibly tiny steps, resulting in a crippled runtime complexity of $O(\epsilon^{-4})$.
- **Over-solving M:** If statistical error is reduced to ϵ^2 by simulating billions of paths, the computational cost skyrockets to $O(\epsilon^{-5})$, while the final result remains bottlenecked by the inherent bias of the large time steps.

To achieve the mathematically optimal baseline complexity of $O(\epsilon^{-3})$, the error budget must be split perfectly evenly. Both the number of paths M and the time step size Δt must scale directly with ϵ, ensuring no CPU cycles are wasted.

3 The Multilevel Monte Carlo Architecture

3.1 Deriving the Maximum Level (L)

The MLMC algorithm simulates paths across a range of resolutions. At level l, the step size is defined as $\Delta t_l = M^{-l}T$ [cite: 138-139]. To determine the maximum required depth L to achieve a target accuracy ϵ, we set the step size at the most refined level equal to the accuracy requirement (dropping constants for Big-O notation):

$$\begin{aligned} M^{-L} &\approx \epsilon \\ \log(M^{-L}) &= \log(\epsilon) \\ -L\log(M) &= \log(\epsilon) \\ L\log(M) &= \log(\epsilon^{-1}) \\ L &= \frac{\log(\epsilon^{-1})}{\log(M)} \end{aligned}$$

This logarithmic relationship [cite: 145] dictates how deep the memory buffers need to be allocated dynamically based on the user's precision requirements.

3.2 The Magic of Path Coupling

The core mechanism of MLMC relies on evaluating the difference in payoffs between a coarse path and a fine path. It is vital that these paths are not independent [cite: 162]; they must come from the exact same discretized Brownian path[cite: 162].

In practice, this is achieved by generating the fine Brownian increments (ΔW_{fine}) first. The coarse increments (ΔW_{coarse}) are then constructed by summing pairs of the fine increments. Because both the fine and coarse Euler-Maruyama approximations are driven by the exact same underlying randomness, strong convergence guarantees they remain tightly bound to each other, resulting in a payoff difference with an incredibly tiny variance.

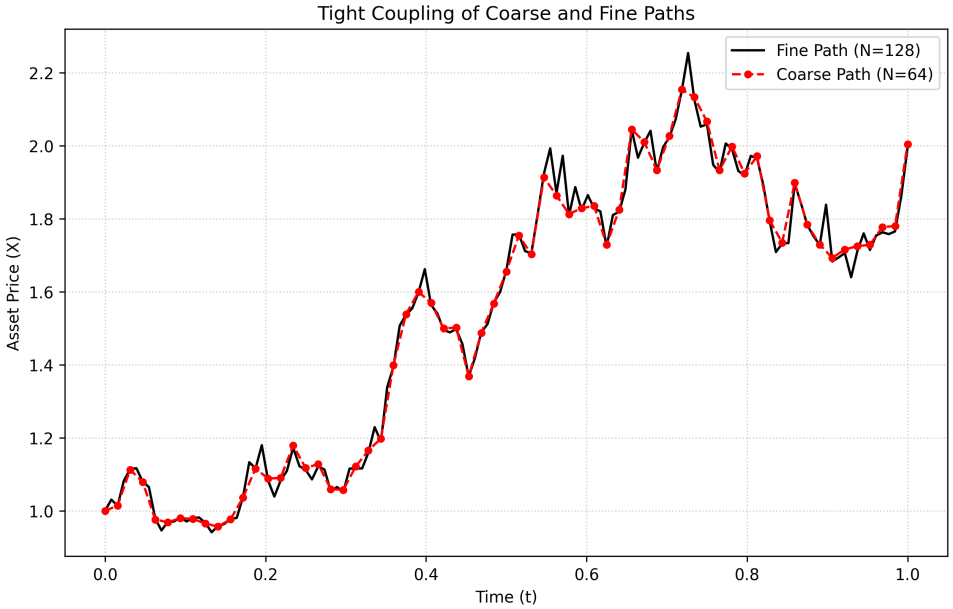

Figure 2: Illustration of tight path coupling. The coarse and fine paths are driven by the same Brownian increments, keeping them physically close.

4 Software Engineering Implementation

Transitioning MLMC from theoretical mathematics to a functional, low-latency engine requires careful attention to system architecture.

The primary bottleneck in SDE simulation is not arithmetic, but memory management. If a simulation engine dynamically allocates a new ‘std::vector‘ for every single path evaluation, the CPU cache will thrash and execution speed will plummet. To mitigate this, the implemented C++ engine utilizes pre-allocated memory buffers. The ‘euler_maruyama’ function signature accepts paths by reference, allowing the hot loop to simply overwrite existing memory blocks, ensuring optimal spatial locality and zero dynamic allocation overhead during the heavy computations.

Furthermore, the architecture is designed to be highly modular. The core solvers utilize ‘std::function‘ templates, allowing arbitrary drift and diffusion coefficients to be passed in, ensuring the engine can price any asset class without altering the underlying mathematics.

5 Empirical Results & Complexity Analysis

5.1 Baseline Validation

To validate the engine, a standard Geometric Brownian Motion asset was priced using parameters from Giles (2008) Section 6 ($X_0 = 100, T = 1, K = 100, r = 0.05, \sigma = 0.25$) [cite: 246-248]. The baseline Standard Monte Carlo simulation yielded a European Call price of \$12.30 and a Digital Call price of \$50.34. The MLMC engine, using the same parameters, produced highly consistent results of \$12.33 and \$50.40, respectively, validating the mathematical accuracy of the implementation.

5.2 Euler-Maruyama Convergence Proof

To confirm the underlying mechanics required for MLMC, the convergence of the Euler-Maruyama scheme was empirically tested.

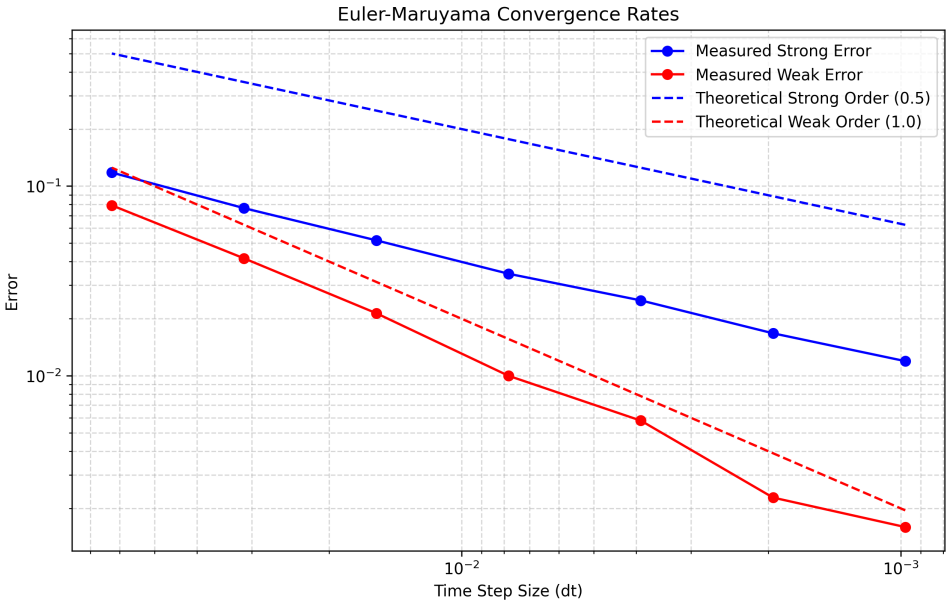

Figure 3: Log-log plot demonstrating the empirical convergence rates of the Euler-Maruyama method.

As shown in the graph above, as the time step Δt shrinks, the weak error scales linearly ($O(\Delta t)$), while the strong error scales proportionally to the square root ($O(\Delta t^{1/2})$), exactly aligning with the theoretical bounds necessary to prove the variance reduction of MLMC.

5.3 The Complexity Benchmark and the "Constant Factor" Reality

The ultimate proof of the MLMC algorithm is its asymptotic time complexity. Standard Monte Carlo scales at $O(\epsilon^{-3})$ [cite: 98], while MLMC scales optimally at $O(\epsilon^{-2}(\log \epsilon)^2)$[cite: 107]. This scaling was benchmarked on physical hardware.

The benchmark data highlights the reality of algorithm design: theoretical complexity ignores constant factors (C). At low accuracies ($\epsilon = 0.05$), MLMC is actually *slower*. This is because simulating one path in MLMC requires significantly more CPU overhead (generating fine increments, summing for coarse increments, and simulating two separate SDE paths) compared to the simple, single loop of Standard MC.

However, as the accuracy requirement tightens, the superior mathematical scaling overtakes the constant overhead. When ϵ was halved from 0.01 to 0.005, the Standard MC execution

Target Accuracy (ϵ)	Standard MC Time (s)	MLMC Time (s)	Speedup Factor
0.05	0.13	0.28	0.46x (Slower)
0.02	1.61	2.34	0.68x (Slower)
0.01	12.19	11.45	1.06x (Faster)
0.005	94.89	56.42	1.68x (Faster)

Table 1: Benchmark execution times comparing Standard Monte Carlo to MLMC.

time exploded from 12 seconds to nearly 95 seconds—a roughly 8x multiplier consistent with $O(\epsilon^{-3})$. Conversely, MLMC only grew from 11.45s to 56s—a roughly 5x multiplier consistent with $O(\epsilon^{-2}(\log \epsilon)^2)$. As $\epsilon \to 0$, this exponential divergence results in massive, scalable time savings.

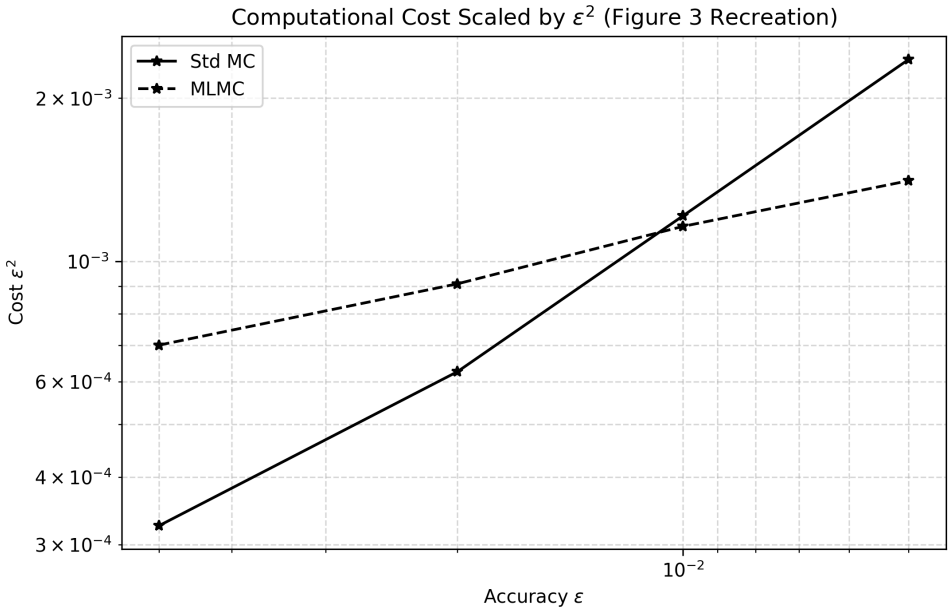

Figure 4: Computational cost scaled by ϵ^2. The flat MLMC line proves its superior asymptotic complexity.

6 Conclusion

This project successfully implemented the Multilevel Monte Carlo algorithm, demonstrating its profound impact on computational finance. The transition from theoretical mathematics to a functional C++ codebase required careful navigation of mathematical assumptions (such as the Lipschitz condition) and hardware realities (such as optimal memory allocation). The empirical benchmarks proved that while sophisticated algorithms introduce higher baseline overhead, their superior asymptotic complexity is indispensable for high-precision, institutional-grade quantitative modeling.